

EXPERIMENT NO. 4

DESIGN AND IMPLEMENTATION OF EDGE–CLOUD SYNCHRONIZATION USING LOCAL MQTT BRIDGE (GOOGLE COLAB VERSION)

AIM

To simulate Edge–Cloud synchronization by collecting solar panel power output data at an edge device, synchronizing it to the cloud through a simulated MQTT bridge, and performing real-time analytics using Python in Google Colab.

OBJECTIVES

- Simulate an Edge Device.
- Generate real-time solar panel power output data.
- Synchronize data from Edge to Cloud.
- Store synchronized cloud data.
- Perform stream analytics.
- Detect low power generation.
- Visualize synchronized data.

SOFTWARE REQUIRED

- Google Colab
- Python 3
- pandas
- numpy
- matplotlib

THEORY

Edge Computing processes solar panel data close to the source, reducing communication delay and enabling faster local decisions.

Cloud Computing receives synchronized data through a simulated MQTT bridge, stores historical records, performs analytics, and provides long-term monitoring of solar panel performance.

Data Flow

Solar Panel



Edge Device



MQTT Bridge (Simulation)



Cloud Storage



Analytics Dashboard

Algorithm

1. Start Program.
2. Generate Solar Panel Power Output.
3. Store data in Edge Memory.
4. Synchronize data to Cloud.
5. Calculate Mean.
6. Calculate Moving Average.
7. Calculate Standard Deviation.
8. Detect Low Power Output.
9. Display Graph.
10. Save CSV File.
11. Stop Program.

GOOGLE COLAB CODE

```
import random

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from IPython.display import clear_output

import time

edge_storage = []

cloud_storage = []

time_data = []

for i in range(1,51):

    power = random.randint(50,350)

    edge_storage.append(power)

    cloud_storage.append(power)

    time_data.append(i)

    mean = np.mean(cloud_storage)

    std = np.std(cloud_storage)

    if len(cloud_storage) >= 5:

        moving = np.mean(cloud_storage[-5:])

    else:

        moving = mean

    if power < 100:

        status = "LOW POWER OUTPUT"

    else:

        status = "NORMAL"

    clear_output(wait=True)

    print("===== EDGE DEVICE =====")

    print("Current Power Output :", power, "W")

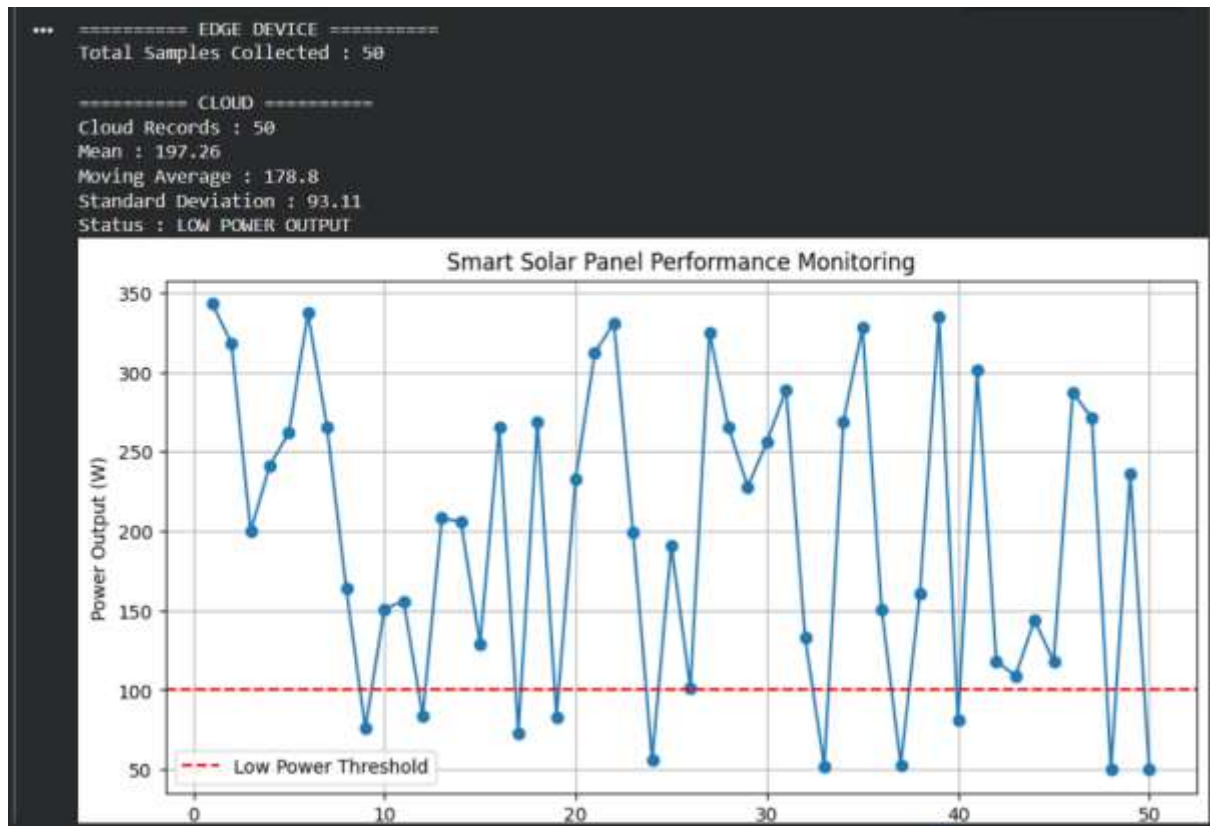
    print("Stored at Edge :", len(edge_storage))
```

```

print()
print("===== CLOUD =====")
print("Cloud Records :", len(cloud_storage))
print("Mean :", round(mean,2))
print("Moving Average :", round(moving,2))
print("Standard Deviation :", round(std,2))
print("Status :", status)
plt.figure(figsize=(10,4))
plt.plot(time_data, cloud_storage, marker='o')
plt.axhline(100, color='red', linestyle='--', label='Low Power Threshold')
plt.xlabel("Sample")
plt.ylabel("Power Output (W)")
plt.title("Edge-Cloud Synchronization")
plt.legend()
plt.grid(True)
plt.show()
time.sleep(0.5)
df = pd.DataFrame({
    "Sample": time_data,
    "Power Output (W)": cloud_storage
})
df.to_csv("cloud_solar_panel_data.csv", index=False)
print("Experiment Completed Successfully")

```

output



RESULT

The edge device successfully collected solar panel power output data and synchronized it with the cloud using a simulated MQTT bridge. The cloud performed real-time analytics using Mean, Moving Average, and Standard Deviation, detected low power output conditions, visualized the synchronized data, and stored the records in a CSV file for future analysis.